

UVME_{env}

Open-source framework to create UVM environments based on Cocotb and PyUVM

MSc. Manuel Emilio Benítez Morales

Circuify Semiconductors - CIC

2026-07-23

What is

UVMEEnv is an open-source and console based framework to be able to create full UVM environments easily and ready to use/develop.

Philosophy (1)

- Easy and agile creation of UVM environments using Python.
- Easy to use/understand for new users but useful for experimented ones
- Congruent, consistent and sustainable architecture between abstract design and filesystem tree.
- Keep good practices on naming, coding, component organization, UVM standard usage, software architecture, etc.
- Make an easy but very effective name reference between components and projects.

- Focus the user on verification process and not on environment creation/configuration.
- High project parametrization and interactivity for decreasing manual coding.
- Auto-handled control signals and behavior according to combinatorial and sequential designs.
- Easy usage of tool, be able to do/handle everything since project root path.

- Make good UVM base environments for very simple and very complex designs.
- Make a fully reusable environment/project, since perspective UVM standard and since perspective of software development.

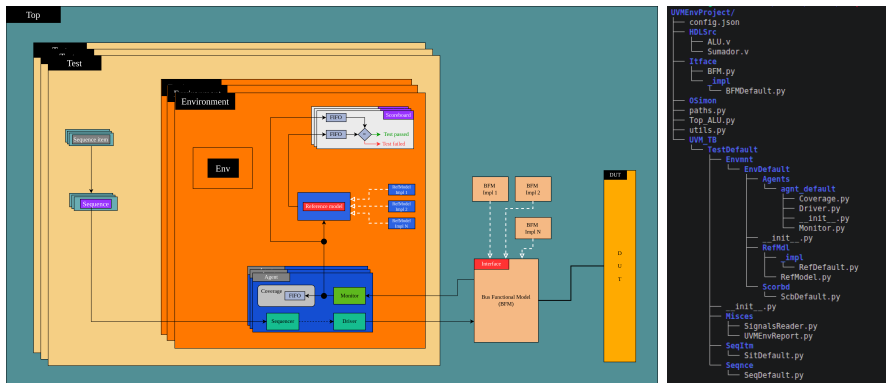
Philosophy (4)

- Make tool for decreasing usage complexity and increasing capabilities.
- Leverage all tools provided on Python language.

Software architecture (1)

Python

Clean architecture



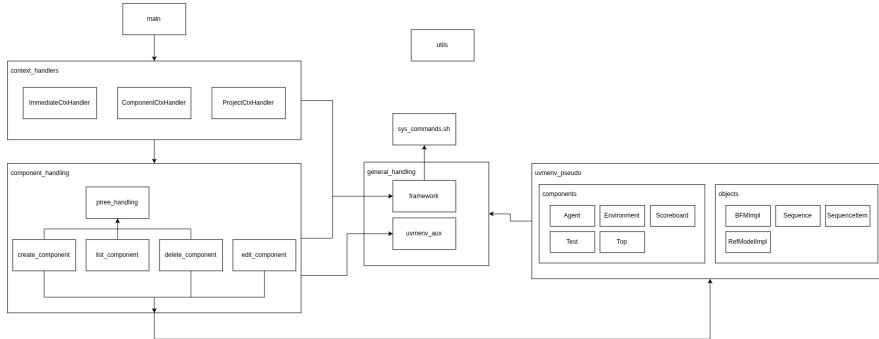
Scope

UVM environment; UVM functionality; Environment components usage

Software architecture (2)

C++

Header/Source Split ; Modular architecture based on feature-driven ; UVM-based pseudo-factory



Scope

Environment handling; Start of simulation; Code writing; Read/write signal names/lengths/types.

Current capabilities (1)

From version 1.0

- Fast UVM environment creation.
- UI to easy project management.
- .csv file generated for faster signal information reading.
- Components handling helped with .yaml file to describe the current state of project.
- Configuration file to customize behavior, instances, names and tools used on project.
- Immediate Top, utils and paths file to handle the scope and utilities usage on project.
- Creation of multiple components: test, environment, scoreboard, agent.
- Creation of polymorphic reference model into each environment.
- Creation of polymorphic BFM but this will be changed to different BFM interfaces (not singleton).

Current capabilities (2)

From version 1.0

- Support for environment into another but not yet managed with UI.
- Support for any kind of RTL code. Can be in a single file, divided files or subdirectories made, you need only put it into HDLSrc.
- Directory focused on save simulation results, currently coverage resume, UVM report and wave dump on VCD format.
- Pre-structured directory tree to ensure consistence on all the environment. Based on clean architecture to have a clear idea.
- Automatic writing of files (see files classification), including signal names, keeping coherence and well relation between components.
- Leverage Python characteristics to reduce code on sequence bodies.
- Connection of external technologies for reference model using th polymorphic reference model. By default you can have a Python or Verilog/SystemVerilog model (which must be compiled into .so file).

Current capabilities (3)

On development

- Make BFM as different objects and not polymorphic.
- Creation per module (currently all generated signal names for components creation is based on top module only).
- Handling for Miscellaneous files (currently available list only).
- Use names for control signals (clock and reset) since configuration file to avoid the needing of edit these names on code.
- Add again support for “refresh” option when loading signals.
- Support for adding external reference models connections by using UI.
- Easy change of simulation tool through configuration file.
- Use new characteristics of Cocotb 2.0 to make base files.

Coexistence with SystemVerilog-